

JAVA LAB WEEK 5

1. Basic Information

Title: Bank Account System Using OOP Concepts

Experiment Topic: Bank Account System using Object-Oriented Programming

Required Java/OOP Concepts:

- Classes and Objects
 - Constructors
 - Inheritance
 - Method Overriding
 - Methods and Attributes
 - super keyword
 - Basic Polymorphism concept
-

2. Objective / Aim

Aim

To develop a simple **Bank Account System** using Java and demonstrate important **Object-Oriented Programming concepts** such as classes, objects, constructors, inheritance, and method overriding.

Objective

The program creates Savings and Current bank accounts and performs basic operations such as:

- Displaying account details
 - Depositing money
 - Withdrawing money
 - Adding interest to a savings account
 - Transferring money between accounts
 - Using inheritance to create specialized account types
 - Overriding the withdrawal method in the Current Account class
-

3. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum 1 GHz Processor
- Keyboard and Monitor

Software Requirements

- Operating System: Windows / Linux / macOS
 - Java Development Kit (JDK): **JDK 8 or above**
 - Java Compiler: javac
 - IDE: **Eclipse / IntelliJ IDEA / NetBeans / VS Code / BlueJ**
 - Command Prompt or Terminal
-

4. Algorithm / Logic Description

1. Start the program.
2. Create a base class named **Account**.
3. Declare common account attributes:
 - Account number
 - Account holder name
 - Balance
 - Account type
4. Create a constructor in the Account class to initialize these attributes.
5. Define methods in the Account class:
 - deposit() to add money
 - withdraw() to remove money
 - transfer() to transfer money to another account
 - show() to display account details
6. Create a class named **Savings** that inherits from Account.

7. Use the super keyword to call the parent class constructor.
8. Add an attribute called rate to store the interest rate.
9. Create an interest() method to calculate and add interest to the savings balance.
10. Create a class named **Current** that inherits from Account.
11. Add an attribute called od to store the overdraft limit.
12. Override the withdraw() method in the Current class so that the account can withdraw money up to the available balance plus the overdraft limit.
13. In the main() method, create:
 - One Savings object
 - One Current object
14. Display the initial details of both accounts.
15. Perform the following Savings Account operations:
 - Deposit ₹2000
 - Withdraw ₹1000
 - Calculate and add 5% interest
16. Perform the following Current Account operations:
 - Deposit ₹3000
 - Withdraw ₹9000
17. Transfer ₹2000 from the Savings Account to the Current Account.
18. Display the final account details.
19. Stop the program.

OOP Concepts Used

Class: Account, Savings, Current, and BankSystem

Objects: s and c

Constructor: Used to initialize account details.

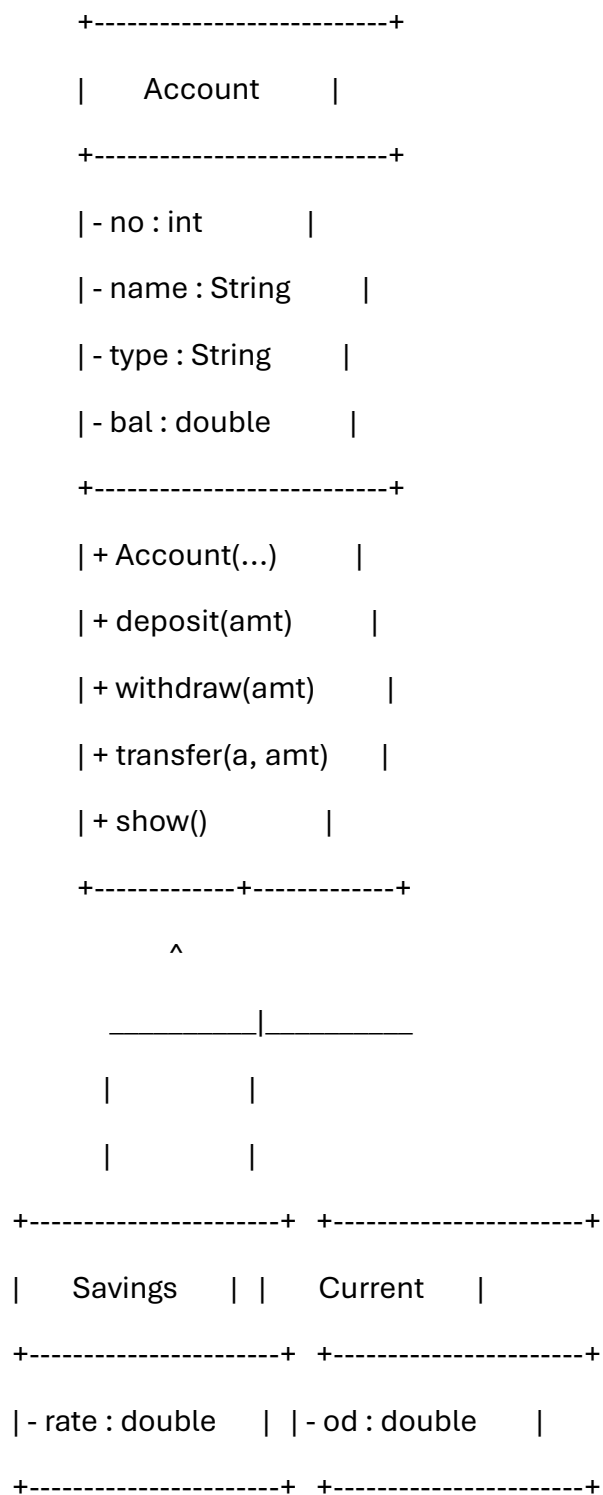
Inheritance: Savings and Current inherit common properties and methods from Account.

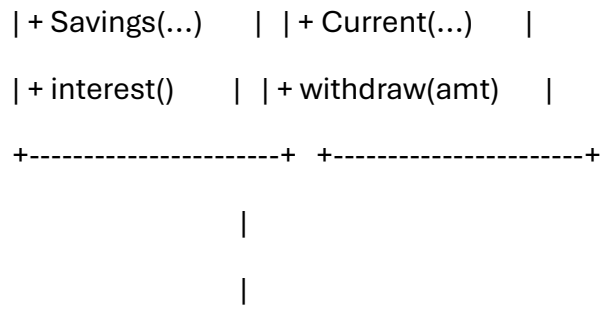
Method Overriding: The withdraw() method of Current overrides the method of Account.

Polymorphism: Method overriding provides different withdrawal behavior for different account types.

super Keyword: Used in child class constructors to call the parent class constructor.

5. UML Class Diagram

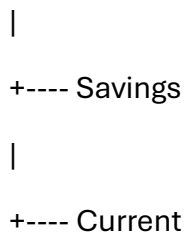




Method Overriding

Class Relationship

Account



Savings and Current are child classes of the Account class.

6. Source Code

Java Filename: BankSystem.java

```

class Account {
    int no;
    String name, type;
    double bal;

    Account(int no, String name, double bal, String type) {
        this.no = no;
        this.name = name;
        this.bal = bal;
        this.type = type;
    }
}
  
```

```
void deposit(double amt) {  
    if (amt > 0) {  
        bal += amt;  
        System.out.println("Deposited: " + amt);  
    }  
}
```

```
void withdraw(double amt) {  
    if (amt > 0 && amt <= bal) {  
        bal -= amt;  
        System.out.println("Withdrawn: " + amt);  
    } else {  
        System.out.println("Insufficient balance");  
    }  
}
```

```
void transfer(Account a, double amt) {  
    if (amt > 0 && amt <= bal) {  
        bal -= amt;  
        a.bal += amt;  
        System.out.println("Transferred: " + amt);  
    } else {  
        System.out.println("Transfer failed");  
    }  
}
```

```
void show() {  
    System.out.println("No    : " + no);  
}
```

```
        System.out.println("Name  :" + name);  
        System.out.println("Type  :" + type);  
        System.out.println("Balance: " + bal);  
    }  
}
```

```
class Savings extends Account {  
    double rate;  
  
    Savings(int no, String name, double bal, double rate) {  
        super(no, name, bal, "Savings");  
        this.rate = rate;  
    }  
  
    void interest() {  
        double in = bal * rate / 100;  
        bal += in;  
        System.out.println("Interest: " + in);  
    }  
}
```

```
class Current extends Account {  
    double od;  
  
    Current(int no, String name, double bal, double od) {  
        super(no, name, bal, "Current");  
        this.od = od;  
    }  
}
```

```

@Override

void withdraw(double amt) {

    if (amt > 0 && amt <= bal + od) {

        bal -= amt;

        System.out.println("Withdrawn: " + amt);

    } else {

        System.out.println("Overdraft limit exceeded");

    }

}

}

public class BankSystem {

    public static void main(String[] args) {

        Savings s = new Savings(102, "Manu", 100000, 5);

        Current c = new Current(103, "Bennu", 50000, 3000);

        System.out.println("=== SAVINGS ===");

        s.show();

        s.deposit(2000);

        s.withdraw(1000);

        s.interest();

        System.out.println("\n=== CURRENT ===");

        c.show();

        c.deposit(3000);

        c.withdraw(9000);

```

```
        System.out.println("\n=== TRANSFER ===");
        s.transfer(c, 2000);

        System.out.println("\n=== FINAL ===");
        s.show();
        c.show();
    }
}
```

7. Compilation & Execution Commands

Step 1: Save the Program

Save the Java program with the filename:

BankSystem.java

Step 2: Compile the Program

Open Command Prompt or Terminal and type:

```
javac BankSystem.java
```

If there are no errors, the program is compiled successfully.

Step 3: Execute the Program

```
java BankSystem
```

8. Output Screen

The output of the program is:

```
=== SAVINGS ===
```

```
No    : 102
```

```
Name  : Manu
```

```
Type  : Savings
```

```
Balance: 100000.0
```

Deposited: 2000.0

Withdrawn: 1000.0

Interest: 5050.0

=== CURRENT ===

No : 103

Name : Bennu

Type : Current

Balance: 50000.0

Deposited: 3000.0

Withdrawn: 9000.0

=== TRANSFER ===

Transferred: 2000.0

=== FINAL ===

No : 102

Name : Manu

Type : Savings

Balance: 104050.0

No : 103

Name : Bennu

Type : Current

Balance: 46000.0

Output Calculation

Savings Account:

Initial Balance = 100000

Deposit = 2000

Balance = 102000

Withdrawal = 1000

Balance = 101000

Interest = $101000 \times 5 / 100$

= 5050

Balance = 106050

Transfer to Current = 2000

Final Savings Balance = $106050 - 2000$

= 104050

Current Account:

Initial Balance = 50000

Deposit = 3000

Balance = 53000

Withdrawal = 9000

Balance = 44000

Received from Savings = 2000

Final Current Balance = $44000 + 2000$

= 46000

Therefore, the final balances displayed by the program are:

Savings Account : 104050.0

Current Account : 46000.0

9. Result

The **Bank Account System using OOP concepts** was successfully implemented in Java.

The program demonstrates **classes, objects, constructors, inheritance, method overriding, the super keyword, and basic polymorphism**. It successfully performs deposit, withdrawal, interest calculation, fund transfer, and account detail display operations.